# `linstab2d`: compressible stability and resolvent analysis with PETSc/SLEPc

Gabriel Hamada Ricardo Frantz

**Abstract**

This paper presents `linstab2d`, an open-source Python implementation of compressible linear stability and resolvent analysis that couples the mathematical framework of Martini & Schmidt (2024) with PETSc/SLEPc sparse solvers. We document the full formulation implemented in the code: the five-variable compressible linearized Navier–Stokes equations (LNSE) for the primitive state vector $\mathbf{q}' = [\rho', u', v', w', T']^\top$, discretized with summation-by-parts (SBP) finite differences on curvilinear grids. The mathematical documentation covers the complete operator assembly — including Sutherland viscosity, the spatial companion linearization for the quadratic eigenvalue problem, the Chu energy norm for compressible disturbance energy, Floquet decomposition for streamwise-periodic base flows, and high-order implicit Padé-type filtering — at the level of detail needed to reproduce the implementation. The analysis framework includes temporal and spatial eigenvalue problems and resolvent analysis. Two solver strategies address the dominant computational cost — repeated sparse linear solves of large shifted non-Hermitian systems: a robust MUMPS sparse-direct baseline and a scalable PETSc iterative path with additive Schwarz preconditioning. The paper defines validation and benchmark protocols for agreement with the MATLAB reference and for large sparse runs. Quantitative accuracy and performance claims are restricted to the cases whose tables and figures are explicitly populated in Section 5. The implementation is released as open-source Python with no MATLAB dependency.

**Keywords:** compressible stability; resolvent analysis; linearized Navier–Stokes; summation-by-parts finite differences; Chu energy norm; PETSc; SLEPc; MUMPS

## 1 Introduction

Linear stability and resolvent analysis are cornerstones of modern transition prediction and turbulence modeling in compressible flows. The mathematical foundations — eigenvalue decomposition for modal instabilities and input–output (resolvent) analysis for nonmodal amplification — are well established [1–3]. Their application to two-dimensional and global configurations has been reviewed by Theofilis [4], while Taira et al. [5, 6] provide a broader perspective on modal analysis techniques in fluid mechanics, including connections to data-driven methods.

A central development in this area is the resolvent framework introduced by McKeon & Sharma [7] and further developed by Towne et al. [8], which recasts the linearized dynamics as a transfer operator between forcing and response. This viewpoint unifies classical stability theory with nonmodal growth and has become a standard tool for analyzing turbulent flows. For compressible flows specifically, the resolvent formulation requires careful treatment of the

energy inner product, where the Chu norm [9, 10] provides a physically meaningful measure of disturbance energy.

The MATLAB implementation of Martini & Schmidt [11] (hereafter M&S) provides a well-documented reference for compressible stability and resolvent analysis on structured grids, including curvilinear coordinate transformations, summation-by-parts (SBP) finite-difference operators, and implicit spatial filtering. Several other tools address related problems: `resolvent4py` [12] provides a Python framework for resolvent and modal decomposition, while the one-way Navier–Stokes (OWNS) approach [13] offers an efficient alternative for convectively dominated flows.

The contribution of the present work is twofold. First, we provide a complete open-source Python implementation of the M&S mathematical framework, including elements not commonly documented in detail: the full five-variable compressible LNSE with Sutherland viscosity, the spatial eigenvalue companion linearization, Floquet-shifted operators for streamwise-periodic flows, the Chu energy norm in discretized form, and high-order implicit Padé-type filtering. Second, we couple this formulation to PETSc/SLEPc [14, 15] for transparent control of sparse linear algebra, offering both a robust MUMPS direct-solve baseline [16] and a scalable iterative path [17, 18] for large-DOF regimes. A comprehensive review of Krylov methods for large-scale stability and resolvent computations in fluid dynamics is given by Frantz et al. [19]; the companion library `LightKrylov` [20] provides a modern Fortran implementation of these techniques.

The remainder of this paper is organized as follows. Section 2 presents the governing equations and analysis framework. Section 3 details the numerical discretization, including SBP operators, curvilinear mappings, boundary treatment, and spatial filtering. Section 4 describes the PETSc/SLEPc solver architecture. Section 5 presents validation and performance results. Section 6 discusses trade-offs and practical guidance, and Section 7 concludes.

# 2 Governing equations and analysis framework

## 2.1 Compressible linearized Navier–Stokes equations

We consider the compressible Navier–Stokes equations for a calorically perfect gas, linearized about a steady base flow $\bar{\mathbf{q}}(\mathbf{x})$. The flow state is decomposed as

$$\mathbf{q}(\mathbf{x}, t) = \bar{\mathbf{q}}(\mathbf{x}) + \epsilon\, \mathbf{q}'(\mathbf{x}, t), \qquad 0 < \epsilon \ll 1, \tag{1}$$

where the perturbation state vector in primitive variables is

$$\mathbf{q}' = [\rho',\, u',\, v',\, w',\, T']^\top. \tag{2}$$

Here $\rho$ is density, $(u, v, w)$ are the velocity components in the $(x, y, z)$ directions, and $T$ is temperature.

All quantities are non-dimensionalized by a reference length $L^*$, velocity $U^*$, density $\rho^*$, and temperature $T^*$, yielding three governing parameters: the Reynolds number $Re = \rho^* U^* L^*/\mu^*$, the Mach number $Ma = U^*/a^*$ (where $a^* = \sqrt{\gamma R_g T^*}$ is the reference sound speed), and the Prandtl number $Pr = \mu^* c_p/\kappa^*$. The equation of state for a perfect gas gives the non-dimensional

pressure as

$$p = \frac{\rho\, T}{\gamma\, Ma^2}. \tag{3}$$

The dynamic viscosity follows Sutherland's law:

$$\mu(T) = \mu_{\mathrm{ref}}\, \frac{T_{\mathrm{ref}} + S_1}{T + S_1} \left(\frac{T}{T_{\mathrm{ref}}}\right)^{3/2}, \qquad S_1 = \frac{110.4\ \mathrm{K}}{T^*}, \tag{4}$$

Here $T_{\mathrm{ref}} = T^*_{\mathrm{ref}}/T^*$ is the nondimensional Sutherland reference temperature, and $\mu_{\mathrm{ref}} = \mu(T^*_{\mathrm{ref}})/\mu^*$ is the corresponding nondimensional reference viscosity. If the nondimensionalization uses $T^* = T^*_{\mathrm{ref}}$ and $\mu^* = \mu(T^*_{\mathrm{ref}})$, then $T_{\mathrm{ref}} = 1$ and $\mu_{\mathrm{ref}} = 1$. The bulk viscosity follows the Stokes hypothesis, $\lambda = -\frac{2}{3}\mu$, and thermal conductivity is determined by $Pr$.

Following M&S [11], the linearized system is written in matrix form as

$$\mathbf{M}\, \frac{\partial \mathbf{q}'}{\partial t} = (\mathbf{A}_0 + \mathbf{A}_x\, D_x + \mathbf{A}_y\, D_y + \mathbf{A}_{xx}\, D_{xx} + \mathbf{A}_{yy}\, D_{yy} + \mathbf{A}_{xy}\, D_{xy})\, \mathbf{q}', \tag{5}$$

where $D_x$, $D_y$, $D_{xx}$, $D_{yy}$, and $D_{xy}$ denote the first and second partial derivative operators, and the $5 \times 5$ coefficient matrices $\mathbf{A}_k$ depend on the base-flow variables and their gradients. The matrix $\mathbf{M}$ accounts for the density weighting of the primitive-variable momentum and energy equations; for Cartesian two-dimensional configurations it reduces to $\mathbf{M} = \mathrm{diag}(1, \bar{\rho}, \bar{\rho}, \bar{\rho}, \bar{\rho})$, while for axisymmetric flows it is a lower-triangular matrix that additionally performs the conservative-to-primitive variable transformation. The explicit expressions for $\mathbf{A}_k$ are given in M&S [11]; we adopt their notation throughout. The original coefficient matrices were derived using computer algebra (Mathematica) by Martini & Schmidt; the present Python implementation was translated from their released MATLAB code. To guard against errors propagated through this derivation chain (Mathematica $\rightarrow$ MATLAB $\rightarrow$ Python), we verify the assembled operator independently using two complementary approaches described in §5.1: symbolic re-derivation with SymPy and a complex-step Jacobian check against the full nonlinear residual.

Key physics encoded in these matrices includes the linearized convection, pressure–density coupling through the equation of state (3), viscous dissipation with the Stokes hypothesis, Sutherland viscosity derivatives (up to second order), and Fourier heat conduction.

**Spanwise Fourier decomposition.** The spanwise ($z$) direction is assumed homogeneous, so perturbations are decomposed into Fourier modes:

$$\mathbf{q}'(x, y, z, t) = \hat{\mathbf{q}}(x, y, t)\, \mathrm{e}^{\mathrm{i}k_z z}, \tag{6}$$

where $k_z$ is the spanwise wavenumber. Under this ansatz, the $z$-derivative operators become algebraic multiplications ($D_z \rightarrow \mathrm{i}k_z$, $D_{zz} \rightarrow -k_z^2$), reducing the three-dimensional problem to a two-dimensional one parameterized by $k_z$. Setting $k_z = 0$ recovers the purely two-dimensional case.

**Axisymmetric formulation.** For axisymmetric base flows, the coordinates $(x, y, z)$ are interpreted as $(x, r, \theta)$, and the azimuthal direction is decomposed into Fourier modes with

integer azimuthal wavenumber $m$: $\mathbf{q}'(x,r,\theta,t) = \hat{\mathbf{q}}(x,r,t)\,\mathrm{e}^{\mathrm{i}m\theta}$. The coefficient matrices $\mathbf{A}_k$ then include the additional metric terms $1/r$ arising from cylindrical geometry, and the matrix $\mathbf{M}$ becomes lower-triangular to perform the conservative-to-primitive transformation. The quadrature weight matrix $\mathbf{W}_\Omega$ includes the radial coordinate $r$ so that inner products integrate over the physical volume element $r\,\mathrm{d}r\,\mathrm{d}x$.

### 2.2 Temporal stability analysis

Temporal stability analysis seeks the complex frequencies $\omega$ at which the linearized system admits exponentially growing or decaying solutions. With the normal-mode ansatz $\mathbf{q}'(\mathbf{x},t) = \hat{\mathbf{q}}(\mathbf{x})\,\mathrm{e}^{-\mathrm{i}\omega t}$, substitution into (5) yields the generalized eigenvalue problem

$$\mathbf{L}\,\hat{\mathbf{q}} = \lambda\,\mathbf{M}\,\hat{\mathbf{q}}, \qquad \lambda = -\mathrm{i}\omega, \tag{7}$$

where $\mathbf{L}$ is the spatially discretized LNSE operator and $\mathbf{M}$ is the discrete mass/conversion matrix. In the implementation, all factors of i are confined to the frequency-domain EVP; after left-multiplication by $\mathbf{M}^{-1}$, the evolution operator is solved in the convention induced by $\mathbf{q}'(\mathbf{x},t) = \hat{\mathbf{q}}(\mathbf{x})\,\mathrm{e}^{-\mathrm{i}\omega t}$. Consequently, *temporally unstable modes have* $\mathrm{Im}(\omega) > 0$, and the phase speed is $c = \mathrm{Re}(\omega)/\alpha$.

For targeted extraction of eigenvalues near a shift $\sigma$, the shift-invert spectral transformation is used:

$$(\mathbf{L} - \sigma\mathbf{M})^{-1}\mathbf{M}\,\mathbf{v} = \mu\,\mathbf{v}, \qquad \mu = \frac{1}{\lambda - \sigma}. \tag{8}$$

Each Krylov iteration of the outer SLEPc eigensolver therefore requires one inner sparse linear solve with the shifted matrix $(\mathbf{L} - \sigma\mathbf{M})$, so the quality of the inner solver directly governs the overall computation [19].

**Adjoint eigenvalues.** The adjoint operator in the $\mathbf{W}$-weighted inner product is

$$\mathbf{L}^{+}_{\mathbf{W}} = \mathbf{W}^{-1}\mathbf{L}^{H}\mathbf{W}, \tag{9}$$

where $\mathbf{W}$ is the diagonal energy-norm weight matrix defined in §2.4. Adjoint eigenvectors are normalized so that $\mathbf{v}^{H}\mathbf{W}\mathbf{v} = 1$ (where $\mathbf{W}$ is real-valued and diagonal).

### 2.3 Spatial stability analysis

Spatial stability analysis seeks the complex streamwise wavenumber $\alpha$ for prescribed real frequency $\omega$. Under the ansatz $\mathrm{e}^{\mathrm{i}\alpha x}$, downstream spatial growth corresponds to $\mathrm{Im}(\alpha) < 0$. Because the LNSE operator (5) depends quadratically on $\alpha$ through the Floquet-shifted derivatives (see §2.5), and the temporal term contributes an additional $\mathrm{i}\omega\mathbf{I}$, the resulting eigenvalue problem is a quadratic eigenvalue problem (QEP):

$$(\mathbf{R}_0 + \mathrm{i}\omega\mathbf{I} + \alpha\,\mathbf{R}_1 + \alpha^2\,\mathbf{R}_2)\,\hat{\mathbf{q}} = \mathbf{0}, \tag{10}$$

where the matrices $\mathbf{R}_0$, $\mathbf{R}_1$, and $\mathbf{R}_2$ are reconstructed from three operator evaluations:

$$\mathbf{R}_0 = \mathbf{A}(0), \qquad \mathbf{R}_1 = \frac{\mathbf{A}(\Delta\alpha) - \mathbf{A}(-\Delta\alpha)}{2\Delta\alpha}, \qquad \mathbf{R}_2 = \frac{\mathbf{A}(\Delta\alpha) - 2\mathbf{A}(0) + \mathbf{A}(-\Delta\alpha)}{2\Delta\alpha^2},$$

with the value of $\Delta\alpha$ reported for each run [21].

The QEP (10) is linearized to a generalized eigenvalue problem of doubled dimension via the companion form. Defining the frequency-free matrix $\mathbf{L}_c$ and the frequency-dependent matrix $\mathbf{L}_{\omega,c}$,

$$\mathbf{L}_c = \begin{bmatrix} \mathbf{0} & \mathbf{I} \\ -\mathbf{R}_0 & \mathbf{0} \end{bmatrix}, \qquad \mathbf{L}_{\omega,c} = \begin{bmatrix} \mathbf{0} & \mathbf{0} \\ -\mathrm{i}\mathbf{I} & \mathbf{0} \end{bmatrix}, \qquad \mathbf{R}_c = \begin{bmatrix} \mathbf{I} & \mathbf{0} \\ \mathbf{R}_1 & \mathbf{R}_2 \end{bmatrix}, \tag{11}$$

the spatial EVP reads

$$(\mathbf{L}_c + \omega\,\mathbf{L}_{\omega,c})\,\mathbf{v} = \alpha\,\mathbf{R}_c\,\mathbf{v}, \qquad \mathbf{v} = \begin{bmatrix} \hat{\mathbf{q}} \\ \alpha\hat{\mathbf{q}} \end{bmatrix}, \tag{12}$$

which is a $2n \times 2n$ generalized non-Hermitian eigenvalue problem (GNHEP) solved using SLEPc with shift-invert and MUMPS.

### 2.4 Resolvent analysis

Resolvent analysis characterizes the amplification between harmonic forcing $\hat{\mathbf{f}}$ and response $\hat{\mathbf{q}}$ at angular frequency $\omega_f$. The transfer (resolvent) operator is

$$\mathbf{H}(\omega_f) = \mathbf{C}_{\mathrm{out}}\,(\mathbf{L} - \mathrm{i}\omega_f\mathbf{I})^{-1}\,\mathbf{B}_{\mathrm{in}}, \tag{13}$$

where $\mathbf{B}_{\mathrm{in}}$ and $\mathbf{C}_{\mathrm{out}}$ are input and output restriction operators that select which state-vector components participate as forcing and response, respectively. In the default (full-state) configuration, $\mathbf{B}_{\mathrm{in}} = \mathbf{C}_{\mathrm{out}} = \mathbf{I}$, so all five primitive variables are both forced and observed; sub-selecting components (e.g., forcing only the momentum equations) is achieved by replacing $\mathbf{B}_{\mathrm{in}}$ or $\mathbf{C}_{\mathrm{out}}$ with rectangular Boolean projection matrices. In the implementation convention used here, the assembled operator $\mathbf{L}$ already includes left multiplication by the inverse conversion matrix from (5); therefore the effective resolvent mass matrix is the identity, giving $\mathbf{L}_\omega = \mathbf{L} - \mathrm{i}\omega_f\mathbf{I}$. The optimal gains $\sigma_j$ and the corresponding forcing–response mode pairs are obtained from the singular value decomposition (SVD) of $\mathbf{H}$ in an energy-weighted inner product.

The compressible disturbance energy is measured using the Chu norm [9], which in non-dimensional primitive variables reads

$$E = \frac{1}{2}\int_\Omega \left[\frac{\bar{T}}{\bar{\rho}\,\gamma\,Ma^2}\,|\rho'|^2 + \bar{\rho}\,(|u'|^2 + |v'|^2 + |w'|^2) + \frac{\bar{\rho}}{\gamma(\gamma-1)\bar{T}\,Ma^2}\,|T'|^2\right] \mathrm{d}\Omega. \tag{14}$$

The corresponding diagonal weight matrix $\mathbf{W} \in \mathbb{R}^{5N\times 5N}$ has blocks

$$\mathbf{W} = \frac{1}{2}\,\mathrm{diag}\left(\frac{\bar{T}}{\bar{\rho}\,\gamma\,Ma^2}\,\mathbf{W}_\Omega,\ \bar{\rho}\,\mathbf{W}_\Omega,\ \bar{\rho}\,\mathbf{W}_\Omega,\ \bar{\rho}\,\mathbf{W}_\Omega,\ \frac{\bar{\rho}}{\gamma(\gamma-1)\bar{T}\,Ma^2}\,\mathbf{W}_\Omega\right), \tag{15}$$

where $\mathbf{W}_\Omega$ is the diagonal matrix of quadrature weights (including the Jacobian determinant for curvilinear grids, and the radial coordinate $r$ for axisymmetric configurations).

For reduced forcing/response spaces, the induced input and output weights are

$$\mathbf{W}_{\text{in}} = \mathbf{B}_{\text{in}}^H \mathbf{W} \mathbf{B}_{\text{in}}, \qquad \mathbf{W}_{\text{out}} = \mathbf{C}_{\text{out}} \mathbf{W} \mathbf{C}_{\text{out}}^H.$$

The energy-weighted SVD is implemented by solving

$$\mathbf{W}_{\text{in}}^{-1} \mathbf{H}^H \mathbf{W}_{\text{out}} \mathbf{H} \mathbf{v}_j = \sigma_j^2 \mathbf{v}_j, \tag{16}$$

where the action of the composed operator $\mathbf{W}_{\text{in}}^{-1} \mathbf{H}^H \mathbf{W}_{\text{out}} \mathbf{H}$ is applied matrix-free via a PETSc `MatShell` and the eigenvalues are extracted by SLEPc's Krylov–Schur eigensolver (`EPS`) targeting the largest-magnitude eigenvalues. Each `MatShell` application requires one forward solve with $\mathbf{L}_\omega = \mathbf{L} - \mathrm{i}\omega_f \mathbf{I}$ and one adjoint solve with $\mathbf{L}_\omega^H$; both reuse a single MUMPS factorization of $\mathbf{L}_\omega$. The adjoint solve requires $\mathbf{A}^H$, which PETSc exposes only as $\mathbf{A}^\top$; the identity

$$\mathbf{A}^\top \mathbf{x} = \overline{\mathbf{A}^H \overline{\mathbf{x}}} \tag{17}$$

bridges the two, and is applied within the transpose solve path. The full-state response mode is recovered as

$$\hat{\mathbf{q}}_j^{\text{full}} = (\mathbf{L} - \mathrm{i}\omega_f \mathbf{I})^{-1} \mathbf{B}_{\text{in}} \mathbf{v}_j / \sigma_j,$$

and the observed response mode is $\hat{\mathbf{q}}_{j,\text{out}} = \mathbf{C}_{\text{out}} \hat{\mathbf{q}}_j^{\text{full}}$.

### 2.5 Floquet ansatz

For streamwise-periodic or slowly varying base flows, a Floquet (Bloch-wave) decomposition replaces the streamwise dependence with a complex exponential modulation:

$$\mathbf{q}'(x, y, t) = \tilde{\mathbf{q}}(x, y)\, \mathrm{e}^{\mathrm{i}\alpha_f x}, \tag{18}$$

where $\alpha_f$ is the Floquet exponent (streamwise wavenumber for parallel flows). Under this ansatz, the streamwise derivative operators are shifted:

$$D_x \to D_x + \mathrm{i}\alpha_f \mathbf{I}, \qquad D_{xx} \to D_{xx} + 2\mathrm{i}\alpha_f D_x - \alpha_f^2 \mathbf{I}. \tag{19}$$

These substitutions are applied per variable block before operator assembly. For parallel base flows, using $\alpha_f = \alpha$ (the streamwise wavenumber) with a single grid point in $x$ recovers the standard one-dimensional eigenvalue problem.

## 3 Numerical discretization

### 3.1 SBP finite differences

Spatial derivatives are approximated using summation-by-parts (SBP) finite-difference operators [22, 23], available at interior orders $2p = 2$, 4, 6, 8. An SBP first-derivative operator $\mathbf{D}_1$ satisfies

$$\mathbf{D}_1 = \mathbf{H}_{\text{sbp}}^{-1} \mathbf{Q}, \qquad \mathbf{Q} + \mathbf{Q}^\top = \mathrm{diag}(-1, 0, \ldots, 0, 1), \tag{20}$$

where $\mathbf{H}_{\text{sbp}}$ is a symmetric positive-definite diagonal norm matrix and $\mathbf{Q}$ is nearly skew-symmetric. Interior stencils use standard $2p$-th order central finite differences, while boundary closures are $p$-th order accurate. Consistent second-derivative operators $\mathbf{D}_2$ are constructed directly (not as $\mathbf{D}_1^2$) to preserve the SBP energy estimate [23].

For axisymmetric configurations, the coordinate singularity at the polar axis is treated using the parity-based pole closures of Mohseni & Colonius [24] (currently implemented for SBP order 2). Ghost points across the axis are reflected with a sign determined by the azimuthal mode number $m$ and the variable parity: for $m = 0$, density, axial velocity, and temperature are symmetric while radial and azimuthal velocities are antisymmetric; for $m = 1$ the roles reverse.

### 3.2 Curvilinear coordinate transformation

The operators are assembled on a structured grid in computational coordinates $(\xi, \eta)$ and mapped to physical coordinates $(x, y)$ via a smooth coordinate transformation with Jacobian

$$\mathbf{J} = \begin{bmatrix} \partial x/\partial \xi & \partial y/\partial \xi \\ \partial x/\partial \eta & \partial y/\partial \eta \end{bmatrix}. \tag{21}$$

Physical-space first derivatives follow from the chain rule:

$$\frac{\partial}{\partial x} = \frac{J_{22}}{|\mathbf{J}|}\frac{\partial}{\partial \xi} - \frac{J_{12}}{|\mathbf{J}|}\frac{\partial}{\partial \eta}, \qquad \frac{\partial}{\partial y} = -\frac{J_{21}}{|\mathbf{J}|}\frac{\partial}{\partial \xi} + \frac{J_{11}}{|\mathbf{J}|}\frac{\partial}{\partial \eta}, \tag{22}$$

where $|\mathbf{J}| = J_{11}J_{22} - J_{12}J_{21}$ is the Jacobian determinant. Second derivatives require additional metric correction terms arising from the non-constancy of the inverse Jacobian; for the $xx$-derivative, for instance,

$$\frac{\partial^2}{\partial x^2} = a^2\frac{\partial^2}{\partial \xi^2} + 2ab\frac{\partial^2}{\partial \xi \partial \eta} + b^2\frac{\partial^2}{\partial \eta^2} + q_\xi\frac{\partial}{\partial \xi} + q_\eta\frac{\partial}{\partial \eta}, \tag{23}$$

where $a = J_{22}/|\mathbf{J}|$, $b = -J_{12}/|\mathbf{J}|$, and $q_\xi$, $q_\eta$ are correction coefficients computed from derivatives of the inverse Jacobian entries. The common special case of a one-dimensional $y$-stretching (the most frequent boundary-layer configuration) simplifies to

$$D_y = \operatorname{diag}(1/y'_\eta)\, D_\eta, \qquad D_{yy} = \operatorname{diag}(1/{y'_\eta}^2)\, D_{\eta\eta} - \operatorname{diag}(y''_\eta/{y'_\eta}^3)\, D_\eta. \tag{24}$$

Quadrature weights for the energy norm are scaled by $|\mathbf{J}|$ to integrate in physical space.

**Shipped grid-stretching maps.** For boundary-layer configurations, the code ships algebraic and hyperbolic-tangent stretching functions that cluster points near a wall while maintaining smoothness. A typical single-sided stretching maps the uniform computational coordinate $\eta \in [0, 1]$ to the physical coordinate $y$ via

$$y(\eta) = y_{\max}\,\frac{1 - \tanh\big[s\,(1-\eta)\big]\,/\tanh(s)}{1 + \tanh\big[s\,(1-\eta)\big]\,/\tanh(s)}, \tag{25}$$

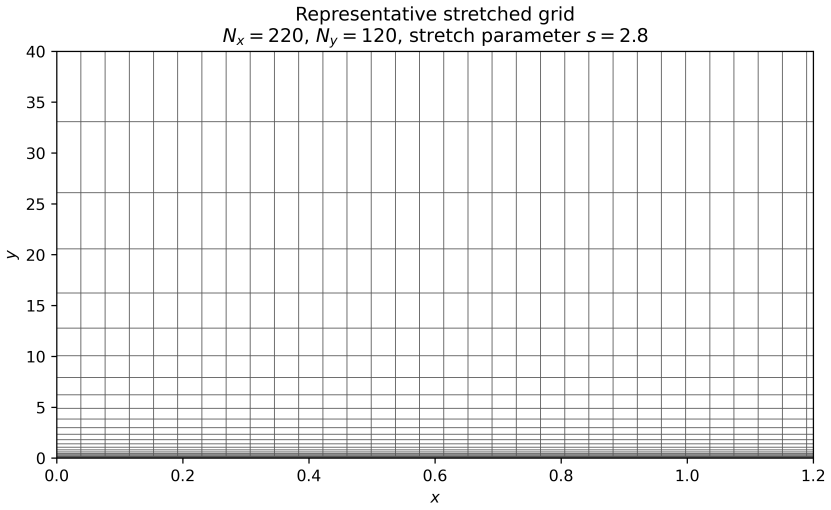



Figure 1: Representative stretched grid for a flat-plate boundary-layer configuration generated with the hyperbolic-tangent mapping in (25). The mesh visibly clusters points near $y = 0$, where near-wall gradients are strongest.

where $s > 0$ controls the clustering intensity and $y_{\max}$ is the domain height. User-supplied grids in physical coordinates are also supported, in which case the Jacobian entries are computed numerically from the SBP operators themselves. Figure 1 shows a representative stretched grid for a boundary-layer configuration.

### 3.3 Boundary conditions

Boundary conditions are imposed via an algebraic boundary overwrite. For homogeneous Dirichlet conditions on constrained degree of freedom $i$, the generalized operator pair is overwritten as

$$A_{i,:} = 0, \qquad A_{:,i} = 0, \qquad M_{i,:} = 0, \qquad M_{:,i} = 0, \qquad A_{ii} = P_{\mathrm{bc}}, \qquad M_{ii} = 1, \tag{26}$$

Neumann (zero normal-gradient) conditions replace only the boundary row with the outward normal derivative computed from the curvilinear metric,

$$L_{i,\cdot} = n_x \, D_{x,i_{\mathrm{mesh}},\cdot} + n_y \, D_{y,i_{\mathrm{mesh}},\cdot}, \tag{27}$$

where $(n_x, n_y)$ is the outward unit normal at the mesh point $i_{\mathrm{mesh}}$. Sponge layers for non-reflecting boundaries are implemented as a spatially varying diagonal damping term added as $\mathbf{L} \leftarrow \mathbf{L} - \mathrm{diag}(\boldsymbol{\sigma}_{\mathrm{sponge}})$. The sponge amplitude follows a smooth polynomial ramp, $\sigma_{\mathrm{sponge}}(d) = \sigma_{\max} \, (d/\ell_s)^p$ for $d \in [0, \ell_s]$, where $d$ is the distance into the sponge region, $\ell_s$ is its thickness, $p$ is the ramp order (default $p = 2$), and $\sigma_{\max}$ is the peak damping rate.

This algebraic overwrite is not the classical SBP-SAT (simultaneous approximation term) formulation; the overwrite parameter $P_{\mathrm{bc}}$ must balance boundary enforcement against linear-system conditioning, as overly large values increase the stiffness of $(\mathbf{L} - \sigma\mathbf{M})$ and can stall iterative Krylov convergence. For Dirichlet conditions, the symmetric overwrite (zeroing both rows and columns) preserves the boundary structure under Hermitian transposition, so the

adjoint operator (9) inherits consistent boundary conditions. Neumann conditions replace the boundary row only (27), so adjoint consistency depends on the specific normal-derivative stencil.

### 3.4 Spatial filtering

High-frequency numerical oscillations are attenuated by a 10th-order Padé-type implicit filter in the framework of Lele [25] and Visbal & Gaitonde [26]. The filtered solution satisfies the implicit relation

$$\mathbf{M}_f \, \mathbf{q}_{\text{filt}} = \mathbf{F} \, \mathbf{q}, \tag{28}$$

where $\mathbf{M}_f$ is a symmetric tridiagonal matrix with unit diagonal and off-diagonal entries equal to the filter parameter $\alpha_{\text{flt}}$,

$$\mathbf{M}_f = \text{tridiag}(\alpha_{\text{flt}},\ 1,\ \alpha_{\text{flt}}), \tag{29}$$

and $\mathbf{F}$ is a banded explicit filter operator with an 11-point interior stencil whose coefficients depend on $\alpha_{\text{flt}}$. The filter strength is controlled by $\alpha_{\text{flt}} \in [0,\, 0.5)$: setting $\alpha_{\text{flt}} = 0$ gives maximum damping of the highest-frequency mode, while $\alpha_{\text{flt}} \to 0.5$ recovers the identity. Five special boundary-closure rows at each end maintain the filter order near domain edges.

For two-dimensional problems, the explicit filter operators along each coordinate are first composed, and the implicit tridiagonal systems are then solved sequentially:

$$\mathbf{q}_{\text{filt}} = \mathbf{M}_{f,y}^{-1} \, \mathbf{M}_{f,x}^{-1} \, \mathbf{F}_x \, \mathbf{F}_y \, \mathbf{q}. \tag{30}$$

This is algebraically equivalent to sequential sweeps $\mathbf{M}_{f,y}^{-1}\mathbf{F}_y \, \mathbf{M}_{f,x}^{-1}\mathbf{F}_x$ when the filter operators are separable along each coordinate direction. An adjoint-consistent filter pair is required for resolvent analysis; the adjoint filter replaces $\mathbf{M}_f^{-1}\mathbf{F}$ with $\mathbf{F}^H\mathbf{M}_f^{-H}$ so that the discrete filter operators satisfy the intended formal adjoint relation in the weighted inner product. The tridiagonal systems (29) are solved using MUMPS LU factorization through PETSc.

### 3.5 Discrete operator assembly

After coordinate transformation and boundary treatment, the discrete LNSE operator $\mathbf{L} \in \mathbb{C}^{n\times n}$ (with $n = 5N$, where $N$ is the number of active grid points) inherits the $5 \times 5$ block structure

$$\mathbf{L} = \begin{bmatrix} \mathbf{L}_{\rho\rho} & \mathbf{L}_{\rho u} & \mathbf{L}_{\rho v} & \mathbf{L}_{\rho w} & \mathbf{L}_{\rho T} \\ \mathbf{L}_{u\rho} & \mathbf{L}_{uu} & \mathbf{L}_{uv} & \mathbf{L}_{uw} & \mathbf{L}_{uT} \\ \mathbf{L}_{v\rho} & \mathbf{L}_{vu} & \mathbf{L}_{vv} & \mathbf{L}_{vw} & \mathbf{L}_{vT} \\ \mathbf{L}_{w\rho} & \mathbf{L}_{wu} & \mathbf{L}_{wv} & \mathbf{L}_{ww} & \mathbf{L}_{wT} \\ \mathbf{L}_{T\rho} & \mathbf{L}_{Tu} & \mathbf{L}_{Tv} & \mathbf{L}_{Tw} & \mathbf{L}_{TT} \end{bmatrix}. \tag{31}$$

For the strictly two-dimensional case $k_z = 0$, the spanwise-velocity equation decouples and the off-diagonal blocks involving $w'$ vanish. For $k_z \neq 0$, continuity, pressure, and viscous terms couple $w'$ to $(\rho', u', v', T')$, so the corresponding blocks are not structural zeros. On unmasked rectangular grids, each $N \times N$ sub-block is assembled efficiently using Kronecker products of the one-dimensional SBP operators. For grids with masked (inactive) points, a line-by-line sparse assembly is used with Fortran-order (column-major) indexing. The operator is stored in

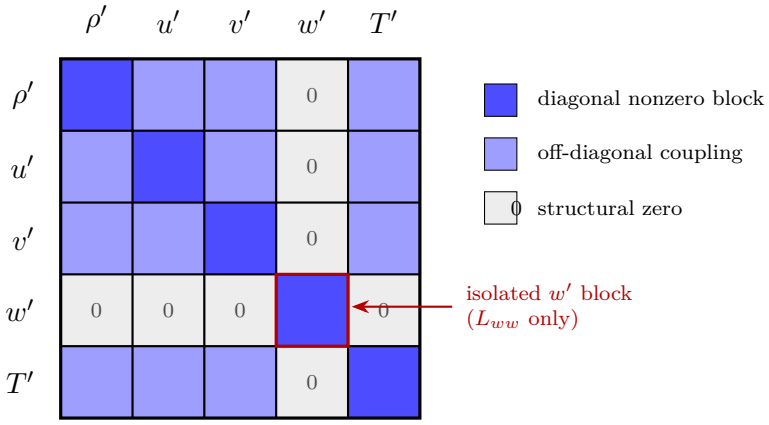



Figure 2: Schematic $5 \times 5$ block structure of the assembled LNSE operator $\mathbf{L}$ for the special case $k_z = 0$. Each cell denotes an $N \times N$ sparse sub-block: dark cells are diagonal blocks, medium cells are off-diagonal couplings, and gray cells marked 0 are structural zeros. For $k_z \neq 0$, the $w'$-coupling blocks become nonzero.

compressed sparse column (CSC) format with complex double precision (`complex128`). Figure 2 illustrates the $5 \times 5$ block sparsity pattern of $\mathbf{L}$ for a representative boundary-layer case.

**Software architecture.** The Python package is organized into three sub-packages: `linstab2d/operators/` contains the coefficient matrices, LNSE assembly, energy-norm construction, and spatial companion linearization; `linstab2d/mesh/` handles grid generation, coordinate transformations, SBP operators, and implicit filters; `linstab2d/solvers/` wraps PETSc/SLEPc for linear solves, eigenvalue problems, and resolvent analysis. A utility layer (`linstab2d/utils/`) provides PETSc initialization, SciPy-to-PETSc conversion, and MPI-aware matrix distribution. Each analysis workflow (temporal, spatial, adjoint, resolvent) is exposed as a single top-level function that accepts SciPy sparse matrices and returns NumPy arrays, hiding the PETSc/SLEPc details from the caller.

# 4 PETSc/SLEPc solver architecture

## 4.1 Design rationale

The dominant computational cost of the analyses described in §2 is repeated solution of large sparse shifted non-Hermitian linear systems: $(\mathbf{L} - \sigma\mathbf{M})\mathbf{x} = \mathbf{b}$ for eigenvalue problems and $(\mathbf{L} + \mathrm{i}\omega\mathbf{M})\mathbf{x} = \mathbf{b}$ for resolvent analysis. PETSc provides a unified interface for sparse matrix storage, iterative or direct linear solvers, and preconditioning, while SLEPc adds outer eigensolver and spectral-transformation layers [14, 15]. In this section, we use PETSc/SLEPc acronyms consistently: `KSP` (linear solver), `PC` (preconditioner), EPS (eigenproblem solver), and ST (spectral transformation). The assembled SciPy CSC operator is converted to a PETSc AIJ matrix once; all subsequent solves operate on the PETSc object. This is the current default implementation path. The target production path is direct PETSc-native distributed assembly to remove temporary SciPy storage pressure for the largest cases.

For problems exceeding the memory of a single node, the assembled operator can be distributed

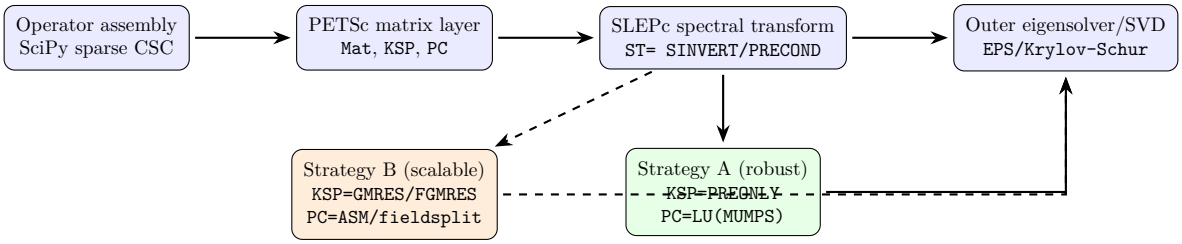



Figure 3: Solver architecture of `linstab2d`. The SciPy CSC operator is converted to PETSc AIJ once, then all downstream operations use PETSc/SLEPc. Dashed arrows denote Strategy B (iterative) and solid arrows denote Strategy A (direct).

across MPI ranks using PETSc's row-partitioned AIJ format. The SciPy operator is assembled on rank 0 and scattered to all ranks, after which all Krylov and direct solves execute in parallel. The matrix distribution mode is selected automatically when `MPI_COMM_WORLD.size > 1`, or can be forced via the environment variable `LINSTAB2D_PETSC_MATRIX_MODE`. Figure 3 illustrates the solver architecture and data flow.

### 4.2 Strategy A: MUMPS sparse-direct

For non-specialist readers, the distinction is:

- **Direct solver:** explicitly factor the sparse matrix, then solve by forward/back substitutions.
- **Iterative solver:** do not factor explicitly; build successive approximations using matrix-vector products and a preconditioner until the residual is below tolerance.

In this workflow, direct methods are usually more robust, while iterative methods are usually more memory-efficient.

MUMPS [16] is used as the default factorization backend via PETSc's `KSP/PC` interface. MUMPS is a sparse multifrontal LU solver: it uses fill-reducing reorderings, assembles dense frontal blocks, and applies partial pivoting for numerical stability. For shift-invert eigensolves, the shifted matrix $(\mathbf{L} - \sigma\mathbf{M})$ is factored once, and each outer Krylov–Schur iteration performs triangular solves with the factors. For resolvent analysis, the MUMPS factorization of $\mathbf{L}_\omega$ supports both forward and transpose solves, enabling the matrix-free application of (16) without re-factoring. This route is preferred when memory permits, as it is maximally robust against non-normality, difficult shifts, and aggressive boundary-penalty scaling. Fill-reducing ordering (METIS, via MUMPS `ICNTL(7)=5`) can be activated to reduce factorization memory. The main limitation remains fill-in growth: factor memory can become prohibitive for very large grids.

### 4.3 Strategy B: preconditioned Krylov

When direct factorization exceeds available memory, a scalable iterative path is used. For a left-preconditioned system,

$$\mathbf{P}^{-1}\mathbf{A}\,\mathbf{x} = \mathbf{P}^{-1}\mathbf{b}, \tag{32}$$

| Criterion | Strategy A: MUMPS sparse-direct | Strategy B: preconditioned Krylov |
|---|---|---|
| Core mechanism | Sparse LU factorization once, then triangular solves for each right-hand side. | Iterative residual reduction with matrix-vector products and preconditioner applications. |
| Robustness | High robustness for difficult non-normal shifted systems. | Strongly dependent on preconditioner quality and shift conditioning. |
| Memory profile | Higher memory demand due to fill-in and factor storage. | Lower memory footprint (no global LU factors). |
| Setup cost | Potentially high one-time factorization cost. | Lower setup cost, but iterative convergence cost per solve. |
| Repeated solves | Efficient after factorization, especially for many solves at fixed shift. | Cost scales with iteration count; can vary across frequencies/shifts. |
| Best-use regime | Default when factors fit in memory and maximum reliability is required. | Preferred when direct factorization is memory-limited and scalable execution is needed. |

Table 1: Reader-oriented comparison of direct and iterative linear-solver strategies used in `linstab2d`.

Krylov iterates are built in the subspace $\mathcal{K}_m(\mathbf{P}^{-1}\mathbf{A}, \mathbf{P}^{-1}\mathbf{r}_0)$. For right preconditioning, one solves

$$\mathbf{A}\mathbf{P}^{-1}\mathbf{y} = \mathbf{b}, \qquad \mathbf{x} = \mathbf{P}^{-1}\mathbf{y},$$

which preserves residual monitoring in the original equation space. In `linstab2d`, the shift-invert eigenvalue inner solve defaults to left preconditioning (GMRES), while the generic iterative linear-solver path defaults to right preconditioning (FGMRES). The former is used for robust fixed-preconditioner shift-invert iterations; the latter is used for flexible Krylov workflows where preconditioner quality may vary across iterations. The default preconditioner is additive Schwarz (ASM) with configurable overlap, a standard domain-decomposition strategy that is robust and MPI-compatible; for readers new to this class of methods, Dolean et al. [27] is an excellent introduction. Increasing overlap often improves convergence robustness, but also increases communication and setup cost. For eigenvalue problems with $k_z = 0$, an optional field-split preconditioner partitions the $(\rho', u', v', T')$ block from $w'$. For $k_z \neq 0$, this split is heuristic rather than exact because the $w'$ equation is coupled to the remaining state variables. This field-split option is activated via the environment variable `LINSTAB2D_EIGEN_PC_TYPE=fieldsplit`. In the current implementation, this field-split path is available for eigenvalue inner solves; resolvent inner solves remain on the ASM-based iterative route. Compared with Strategy A, Strategy B reduces factor-memory pressure because no global LU factors are formed; however, robustness now depends on preconditioner quality and on the conditioning of the shifted operator. All iterative workflows enforce strict failure semantics: a non-converged inner linear solve invalidates the outer eigensolver or resolvent result rather than being silently accepted.

| Layer | Option pattern | Default role | Why chosen | When to change |
|---|---|---|---|---|
| SLEPc | $ST$ = SINVERT | Interior-target extraction | Maps target eigenvalues to dominant transformed eigenpairs | Change shift $\sigma$ when target region moves |
| PETSc KSP/PC | PREONLY + LU(MUMPS) | Strategy A inner solve | Maximum robustness for difficult shifted systems | Replace when factor memory becomes limiting |
| PETSc KSP/PC | GMRES + ASM | Strategy B eigensolver ST | Default iterative inner solve for shift-invert eigenproblems | Switch to fieldsplit via `LINSTAB2D_EIGEN_PC_TYPE` |
| PETSc KSP/PC | FGMRES + ASM | Strategy B resolvent | Flexible GMRES accommodates variable preconditioning | Adjust overlap via `LINSTAB2D_LINSOLVE_ASM_OVERLAP` |
| Conv. guard | `ksp_error_if_not_converged` | Hard failure semantics | Prevents polluted outer spectra from unconverged inner solves | Never disable for production runs |

Table 2: Solver-option rationale used in `linstab2d` for large sparse shift-invert and resolvent workflows.

### 4.4 Solver option summary

Table 2 summarizes the role of the main PETSc/SLEPc configuration options.

All solver parameters are controlled at runtime via environment variables prefixed with `LINSTAB2D_`. The main options are:

- `LINSTAB2D_LINSOLVE_TYPE` and `LINSTAB2D_EIGEN_ST_SOLVER`: select direct (`mumps`) or iterative (`iterative`) mode.
- `*_KSP_TYPE`, `*_PC_TYPE`, `*_KSP_RTOL`, and `*_KSP_MAX_IT`: configure the Krylov solver and preconditioner.
- `*_ASM_OVERLAP`: sets ASM overlap depth.
- `LINSTAB2D_PETSC_MATRIX_MODE`: selects serial or distributed matrix assembly.
- `LINSTAB2D_PETSC_SET_MUMPS_ICNTL_7`: activates METIS fill-reducing ordering.

No dedicated solver configuration file is required; solver tuning is performed through these environment variables.

**Assembly backend and memory pressure.** The current implementation assembles the linear operators in SciPy sparse format and then converts them to PETSc matrices for linear/eigenvalue solves. This design was chosen during migration because it simplifies parity checks against the MATLAB reference and makes operator debugging straightforward. However, for large refined cases it is not the ideal end state: SciPy-first assembly can temporarily duplicate sparse storage and, in root-led workflows, concentrates memory pressure before PETSc distribution. For production-scale runs near the memory limit, the preferred architecture is direct PETSc-native assembly with distributed ownership and preallocation, so the global SciPy operator

can be eliminated from the critical path. The transition strategy is to keep SciPy assembly as a short-term parity/debug mode and make PETSc-native assembly the default path once operator-equivalence checks are complete.

# 5 Results

Results are reported only for cases with archived run artifacts, frozen solver options, and recorded hardware metadata. Results are structured in three layers of increasing scope: first, the linearized operator itself is verified independently of any reference implementation (§5.1); second, full-workflow outputs are compared against the MATLAB reference (§5.2–§5.4); third, solver performance is assessed (§5.5–§5.6). Placeholder tables, workspace-dependent overlays, and TODO captions must be removed before submission.

## 5.1 Independent verification of the linearized operator

The coefficient matrices $\mathbf{A}_k$ in (5) were originally derived with computer algebra (Mathematica) by M&S, implemented in MATLAB, and then translated to Python. Agreement with the MATLAB reference is used to confirm translation fidelity once Table 3 is populated, but this parity check cannot detect errors shared by both implementations. We therefore verify the assembled discrete operator $\mathbf{L}$ against two independent references that do not rely on the M&S derivation.

**Complex-step Jacobian verification.** The discrete nonlinear compressible Navier–Stokes residual $\mathbf{R}(\mathbf{q})$ is evaluated on the same grid and with the same SBP operators used for the linearized system. The exact Jacobian of $\mathbf{R}$ at the base flow $\bar{\mathbf{q}}$ is then computed column-by-column via the complex-step method [28, 29]:

$$\left.\frac{\partial R_i}{\partial q_j}\right|_{\bar{\mathbf{q}}} = \frac{\mathrm{Im}[\mathbf{R}(\bar{\mathbf{q}} + \mathrm{i}\epsilon\, \mathbf{e}_j)]}{\epsilon}, \tag{33}$$

where $\epsilon \sim 10^{-30}$ is a vanishingly small step and $\mathbf{e}_j$ is the $j$-th unit vector. In IEEE double precision, this magnitude is far from underflow and is numerically admissible for complex-step differentiation. Unlike finite differences, complex-step derivatives can reach near machine precision (no subtractive cancellation), provided $\mathbf{R}$ is implemented without non-analytic operations (`abs`, `real`). The resulting Jacobian matrix is compared entry-by-entry against the assembled $\mathbf{L}$; discrepancies above $O(10^{-12})$ indicate coefficient errors. For moderate grids ($n \lesssim 5000$), all $n$ columns are checked; for larger problems, a random subset of columns suffices to provide statistical confidence.

**Symbolic re-derivation with SymPy.** As a complementary check, the compressible Navier–Stokes equations are linearized from scratch using SymPy symbolic algebra. The base-flow and perturbation quantities are defined as SymPy symbols; substitution of $\mathbf{q} = \bar{\mathbf{q}} + \epsilon\mathbf{q}'$ followed by series expansion in $\epsilon$ and collection at $O(\epsilon)$ yields the linearized equations. The coefficient matrices $\mathbf{A}_k$ are extracted by grouping terms according to the derivative operators ($D_x$, $D_y$,

| Case | Grid | $n = 5N$ | $\|\Delta\mathbf{L}\|_F / \|\mathbf{L}_{\text{mat}}\|_F$ | $\|\omega_1^{\text{py}} - \omega_1^{\text{mat}}\|$ | Status |
|---|---|---|---|---|---|
| Blasius ($Re = 1000$) | $1 \times 200$ | pending | pending | pending | pending |
| Compressible BL ($Ma = 2$) | $1 \times 200$ | pending | pending | pending | pending |
| Cylinder ($Re = 100$) | $80 \times 120$ | pending | pending | pending | pending |
| Jet ($Ma = 0.9$, $m = 0$) | $1 \times 300$ | pending | pending | pending | pending |
| Spatial Blasius | $1 \times 200$ | pending | pending | pending | pending |
| Resolvent BL ($\omega_f = 0.1$) | $1 \times 200$ | pending | pending | pending | pending |

Table 3: Accuracy gate reporting template for operator and leading-eigenvalue agreement between `linstab2d` (Python) and the MATLAB reference for all shipped validation cases. Numerical entries are populated by the automated benchmark pipeline (see §5.7).

$D_{xx}$, etc.), and each entry is compared symbolically against the expressions implemented in `LNSECoefficients`. This approach catches algebraic errors in the derivation itself — including Sutherland-viscosity derivative terms, which involve second-order temperature derivatives and are particularly prone to sign or factor errors.

**Verification status.** This subsection defines the independent verification protocol and reporting criteria. Before release snapshots, quantitative values are populated from automated reports, including (i) maximum relative Jacobian discrepancy from the complex-step test and (ii) symbolic agreement status for all coefficient sub-blocks.

### 5.2 Accuracy gate: full shipped validation set

This subsection is validation-only: the goal is to confirm output equivalence with the MATLAB reference, not to claim performance advantages. The accuracy gate consolidates all migrated MATLAB examples into a single table. For each case, we report operator-level agreement (Frobenius-norm relative error of the assembled $\mathbf{L}$), leading eigenvalue and resolvent-gain errors, and a pass/fail gate under predefined tolerances.

Figure 4 overlays the computed eigenvalue spectra from both implementations for a representative case.

Figure 5 compares eigenvector profiles from both implementations.

### 5.3 Resolvent gain validation

For the resolvent workflow, we compare singular values at the exported forcing frequency. Figure 6 shows the modal gain distribution for the representative Hamada case.

### 5.4 Cross-framework comparison

This subsection provides external context for both validation and performance interpretation. Cross-framework comparisons are performed on shared cases with matched physics and discretization whenever possible. Table 4 compares against the legacy MATLAB workflow and `resolvent4py` CGL examples [30]. When solver configurations cannot be made exactly equivalent, results are labeled as approximate and accompanied by full solver/hardware metadata.

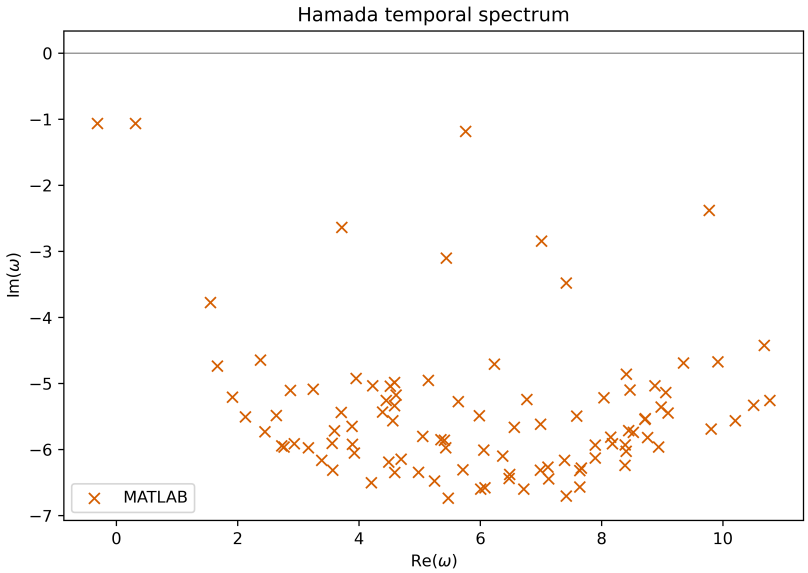



Figure 4: Temporal eigenvalue spectrum for the Hamada reference case on the complex $\omega$-plane. MATLAB exports are shown explicitly, and Python markers are overlaid when the cached run artifact is available in the local workspace.

| Case | Framework | Leading $\omega_1$ or $\sigma_1$ | Rel. error | Wall time [s] | Peak mem [GB] |
|---|---|---|---|---|---|
| Blasius eigen | MATLAB | pending | — | pending | pending |
| | `linstab2d` | pending | pending | pending | pending |
| CGL resolvent | `resolvent4py` [30] | pending | — | pending | pending |
| | `linstab2d` | pending | pending | pending | pending |

Table 4: Cross-framework reporting template for leading eigenvalue/resolvent gain, wall time, and peak memory. Numerical entries are populated by the automated benchmark pipeline (see §5.7).

### 5.5 Flagship large-scale case

This subsection focuses on production-style scalability behavior on a representative large case. The flagship Hamada case stresses the solver stack under production-scale pressure. Both stability and resolvent workflows are reported.

Figure 7 shows the leading eigenvector and resolvent response mode for this case.

### 5.6 Runtime and memory scaling

Performance analysis is reported with hardware and solver metadata (MPI ranks, thread settings, linear-solver/preconditioner choices, tolerances). Table 6 reports wall time and peak memory as a function of problem size for Strategy A and Strategy B.

Figure 8 visualizes the scaling behavior.

From a computational-complexity perspective, direct multifrontal factorization is robust but limited by fill-in growth as mesh refinement increases. Iterative strategies reduce factorization memory pressure, but total time-to-solution depends on preconditioner quality and the non-normal shifted spectrum.

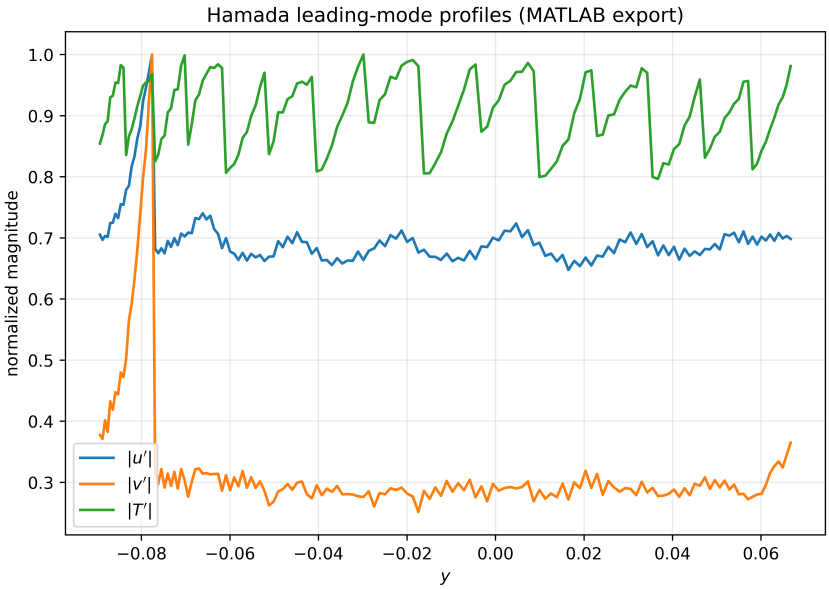



Figure 5: Leading-mode profiles from the Hamada MATLAB export: normalized magnitudes of $|u'|$, $|v'|$, and $|T'|$ along the wall-normal coordinate.

| Workflow | Grid | $n = 5N$ | Leading output | Wall time [s] | Peak mem [GB] |
|---|---|---|---|---|---|
| Temporal EVP | $N_x \times N_y$ | pending | $\omega_1 =$ pending | pending | pending |
| Resolvent | $N_x \times N_y$ | pending | $\sigma_1 =$ pending | pending | pending |

Table 5: Flagship Hamada case reporting template: leading eigenvalue and resolvent gain with solver performance under Strategy A (MUMPS). Hardware and numerical entries are populated by the automated benchmark pipeline (see §5.7).

### 5.7 Reproducibility checklist

All results in this paper are generated with the command pair

```
export LINSTAB2D_LINSOLVE_TYPE=mumps
python -m linstab2d.examples.validate_all --ref matlab_export/
```

which produces `validation_summary.json` in the repository root. The single source of truth for run definitions (cases, expected artifacts, and orthogonality tolerance) is `linstab2d/config/validation_cases.yaml`. Every table and figure in §5 is filled from this summary and recorded with the corresponding Git commit hash.

## 6 Discussion

The primary engineering trade-off lies between the robustness of direct solvers and the scalability of iterative methods. Direct MUMPS runs are reliable when memory permits. Iterative workflows can reduce factorization memory cost but require stronger preconditioning and careful convergence diagnostics, especially for shifted non-normal systems.

Beyond solver selection, the mathematical framework documented in §2–3 enables extensions that were not straightforward in the original MATLAB implementation. The Floquet ansatz (19)

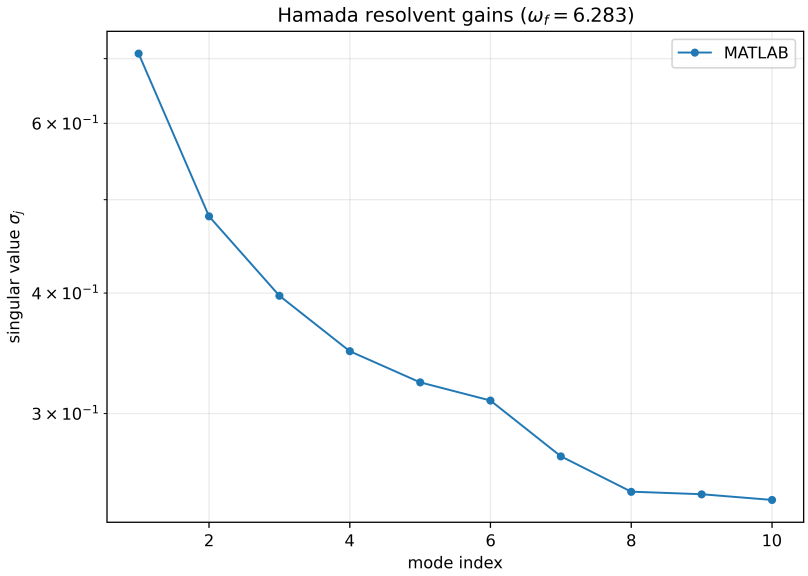



Figure 6: Resolvent singular values $\sigma_j$ versus mode index for the Hamada exported configuration. MATLAB values are shown explicitly, with optional Python overlay when run artifacts are available.

| | Strategy A (MUMPS) | | | Strategy B (FGMRES+ASM) | | |
|---|---|---|---|---|---|---|
| $n = 5N$ | Assemble [s] | Factor [s] | Mem [GB] | Assemble [s] | Solve [s] | Mem [GB] |
| pending | pending | pending | pending | pending | pending | pending |
| pending | pending | pending | pending | pending | pending | pending |
| pending | pending | pending | pending | pending | pending | pending |
| pending | pending | pending | pending | pending | pending | pending |

Table 6: Scaling reporting template: wall time and peak memory vs. problem size for the temporal eigenvalue problem (10 eigenvalues near shift $\sigma$). Hardware and numerical entries are populated by the automated benchmark pipeline (see §5.7).

combined with the spatial companion linearization (12) provides spatial eigenvalues from the same operator assembly. The Chu energy norm (14) in discretized form (15) accounts for the full compressible thermodynamic coupling, and its implementation as a diagonal weight allows efficient matrix-free resolvent SVD via the MatShell interface. The adjoint filter construction in §3.4 preserves the formal discrete adjoint relation of the filter operator; its effect on computed gains must still be checked by filter-on/filter-off sensitivity tests.

The solver-selection workflow can be stated as a reproducible process:

1. assemble and validate operator blocks on the target grid;
2. run Strategy A as the robustness reference if factor memory is feasible;
3. if factor memory is not feasible, switch to Strategy B with ASM preconditioning (or optional field-split for eigenvalue problems) and strict convergence checks;
4. compare outputs against the Strategy A reference on a smaller nearby case before scaling up;
5. only accept production results when inner linear solves meet the prescribed tolerances.

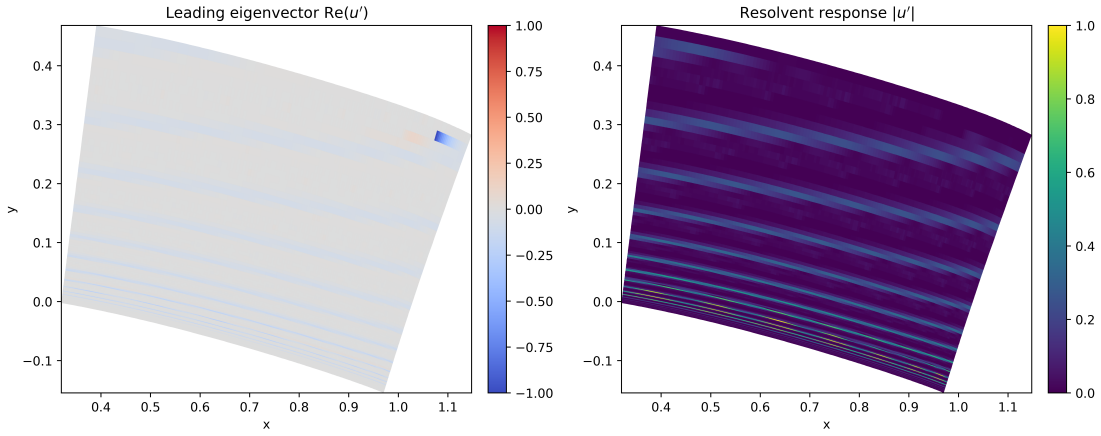



Figure 7: Flagship Hamada case on the $150 \times 170$ grid: (a) normalized real part of the leading temporal eigenvector component $u'$; (b) normalized magnitude of the leading resolvent response component $|u'|$.

**Getting started.** A practical first-use workflow is: (i) install via the shipped `install.sh` script, which handles PETSc/SLEPc and all Python dependencies; (ii) run one of the shipped validation examples to verify the solver stack against the MATLAB reference data; (iii) reproduce a published case to build confidence in the operator assembly; and (iv) scale up to the target configuration with Strategy A, switching to Strategy B only when direct factorization memory is exhausted.

**Known limitations.** The following limitations are documented for transparency:

- Parity-based pole closures for axisymmetric grids are currently implemented only for SBP order 2; higher orders require stencil extensions not yet available.
- The resolvent solver re-factors $\mathbf{L}_\omega$ for each frequency $\omega_f$; no factorization cache is maintained across frequencies.
- The `MatShell` operator in (16) declares a non-Hermitian problem type (`NHEP`) to SLEPc, which is conservative but may forgo Hermitian-specific solver optimizations.
- Field-split preconditioning is available only for the eigenvalue path, not for the resolvent linear solves.

**Assumptions and validity range.**

- Base flow is steady and satisfies the full compressible Navier–Stokes equations to machine precision.
- Grid satisfies the SBP summation-by-parts property by construction.
- Penalty parameter $P_{\mathrm{bc}} = 10^4$ yields boundary errors below $10^{-8}$ on the Blasius validation case.
- Filter strength $\alpha_{\mathrm{flt}} \leq 0.45$ preserves the intended high-order behavior.

**Equation-to-code map.** Table 7 links each key equation to the implementing source file and function, so the paper serves as an annotated code walkthrough. All paths are relative to the package root directory `linstab2d/`.

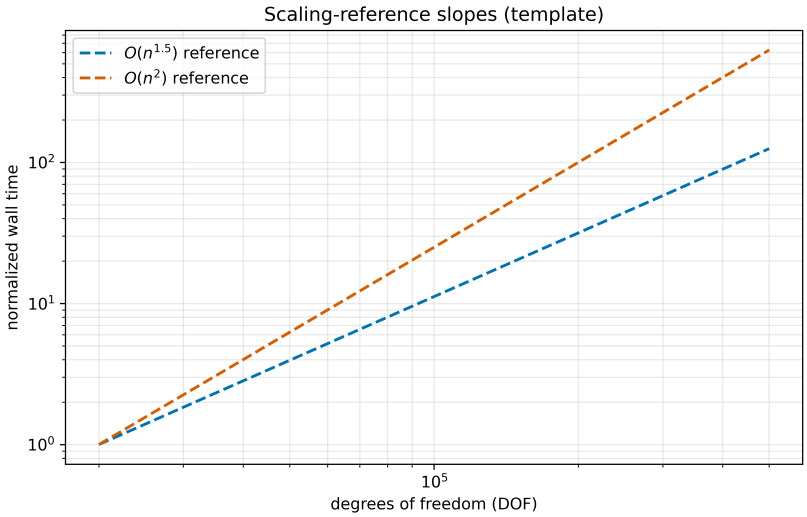



Figure 8: Heuristic reference slopes used to interpret measured scaling data: $O(n^{1.5})$ and $O(n^2)$ on a log-log axis. This figure serves as a template overlay; measured Strategy A/Strategy B wall times are added once the scaling table data in Table 6 are finalized.

| Eq. | Description | File | Entry point |
|---|---|---|---|
| (5) | LNSE matrix form | `operators/coefficients.py` | `LNSECoefficients` |
| (7)–(8) | Temporal EVP | `solvers/eigenvalue.py` | `temporal_eigenvalues` |
| (9) | Adjoint operator | `solvers/eigenvalue.py` | `adjoint_eigenvalues` |
| (10)–(12) | Spatial QEP/companion | `operators/spatial.py` | `build_spatial_operator` |
| (16) | Resolvent SVD | `solvers/resolvent.py` | `resolvent_analysis` |
| (14)–(15) | Chu energy norm | `operators/energy_norm.py` | `build_energy_norm` |
| (19) | Floquet shift | `operators/assembly.py` | `assemble_lnse` |
| (28)–(30) | Implicit filter | `mesh/filters.py` | `build_filter` |

Table 7: Mapping of manuscript equations to source code entry points.

# 7 Conclusions

This paper has presented both the mathematical framework and the solver architecture of `linstab2d`, an open-source Python tool for compressible linear stability and resolvent analysis. The formulation — compressible five-variable LNSE with Sutherland viscosity, SBP finite differences with curvilinear mapping, Chu energy-norm weighting, Floquet decomposition, spatial companion linearization, and Padé implicit filtering — is documented at the level of detail needed to reproduce or extend the implementation. The solver layer exposes both a MUMPS sparse-direct baseline and a preconditioned iterative path; any large-DOF performance claim is restricted to the archived cases reported in Section 5.

The central contribution is the combination of mathematical transparency and solver flexibility in a single open-source package.

Several directions for future work are motivated by the known limitations documented in §6: (i) extending the parity-based pole closures to SBP orders higher than 2, enabling high-order

axisymmetric computations; (ii) implementing a factorization cache across resolvent frequencies to amortize the MUMPS factorization cost when sweeping over $\omega_f$; (iii) declaring the `MatShell` operator in (16) as Hermitian (`HEP`) to unlock SLEPc's Hermitian-specific solver optimizations; (iv) developing stronger iterative preconditioners (e.g., Schur-complement or multigrid variants) for demanding resolvent configurations; and (v) extending field-split preconditioning to the resolvent linear-solve path. Broader parameter studies within the large-DOF regime and systematic comparisons with other frameworks will further establish the practical operating envelope of the tool.

## Code availability

The source code of `linstab2d` is available at:

`https://github.com/ricardofrantz/linstab2d`

The repository includes the full Python package, all validation examples with MATLAB reference data, and an automated installation script (`install.sh`) that handles PETSc, SLEPc, and Python dependencies. The code is released under the open-source license specified in the repository root for each tagged release. A persistent archive is deposited at Zenodo for each public release; the DOI is added to the release metadata and the camera-ready manuscript. A fully reproducible Docker image `ghcr.io/ricardofrantz/linstab2d:paper` contains PETSc, validation data, and the exact scripts used to generate the reported tables and figures.

## Acknowledgments

To be completed.

## Conflict of interest

All authors declare no conflicts of interest in this paper.

## References

[1] Peter J. Schmid and Dan S. Henningson. *Stability and Transition in Shear Flows*. Springer, 2001.

[2] Peter J. Schmid. Nonmodal stability theory. *Annual Review of Fluid Mechanics*, 39:129–162, 2007.

[3] Lloyd N. Trefethen, Anne E. Trefethen, Satish C. Reddy, and Tobin A. Driscoll. Hydrodynamic stability without eigenvalues. *Science*, 261(5121):578–584, 1993.

[4] Vassilis Theofilis. Global linear instability. *Annual Review of Fluid Mechanics*, 43:319–352, 2011.

[5] Kunihiko Taira, Steven L. Brunton, Scott T. M. Dawson, Clarence W. Rowley, Tim Colonius, Beverley J. McKeon, Oliver T. Schmidt, Stanislav Gordeyev, Vassilis Theofilis, and Lawrence S. Ukeiley. Modal analysis of fluid flows: An overview. *AIAA Journal*, 55 (12):4013–4041, 2017.

[6] Kunihiko Taira, Maziar S. Hemati, Steven L. Brunton, Yiyang Sun, Karthik Duraisamy, Shervin Bagheri, Scott T. M. Dawson, and Chi-An Yeh. Modal analysis of fluid flows: Applications and outlook. *AIAA Journal*, 58(3):998–1022, 2020.

[7] Beverley J. McKeon and Ati Sharma. A critical-layer framework for turbulent pipe flow. *Journal of Fluid Mechanics*, 658:336–382, 2010.

[8] Aaron Towne, Oliver T. Schmidt, and Tim Colonius. Spectral proper orthogonal decomposition and its relationship to dynamic mode decomposition and resolvent analysis. *Journal of Fluid Mechanics*, 847:821–867, 2018.

[9] Boa-Teh Chu. On the energy transfer to small disturbances in fluid flow (Part I). *Acta Mechanica*, 1(3):215–234, 1965.

[10] Ardeshir Hanifi, Peter J. Schmid, and Dan S. Henningson. Compressible stability of growing boundary layers using the PSE and comparison with experiment and linear stability theory. *Theoretical and Computational Fluid Dynamics*, 8(3):219–235, 1996.

[11] Eduardo Martini and Oliver Schmidt. Linstab2D: stability and resolvent analysis of compressible viscous flows in MATLAB. *Flow, Turbulence and Combustion*, 2024. Published online 13 August 2024.

[12] Alberto Padovan, Vishal Anantharaman, Clarence W. Rowley, Blaine Vollmer, Tim Colonius, and Daniel J. Bodony. Resolvent4py: An open-source package for stability, resolvent and modal decomposition analyses in python. *SoftwareX*, 31:102286, 2025. doi: 10.1016/j.softx.2025.102286.

[13] Aaron Towne, Adrián Lozano-Durán, and Xiang Yang. Efficient global resolvent analysis via the one-way Navier–Stokes equations. *Journal of Fluid Mechanics*, 948:A9, 2022.

[14] Satish Balay et al. Petsc/tao users manual. *Argonne National Laboratory*, 2025. ANL-95/11 - Revision 3.24.

[15] Vicente Hernandez, Jose E. Roman, and Vicente Vidal. SLEPc: A scalable and flexible toolkit for the solution of eigenvalue problems. *ACM Transactions on Mathematical Software*, 31(3):351–362, 2005.

[16] Patrick R. Amestoy, Iain S. Duff, Jacko Koster, and Jean-Yves L'Excellent. A fully asynchronous multifrontal solver using distributed dynamic scheduling. *SIAM Journal on Matrix Analysis and Applications*, 23(1):15–41, 2001.

[17] Yousef Saad. *Iterative Methods for Sparse Linear Systems*. SIAM, 2 edition, 2003.

[18] Michele Benzi. Preconditioning techniques for large linear systems: a survey. *Journal of Computational Physics*, 182(2):418–477, 2002.

[19] Ricardo A. S. Frantz, Jean-Christophe Loiseau, and Jean-Christophe Robinet. Krylov methods for large-scale dynamical systems: Application in fluid dynamics. *Applied Mechanics Reviews*, 75(3):030802, 2023. doi: 10.1115/1.4056808.

[20] J. Simon Kern, Ricardo S. Frantz, and Jean-Christophe Loiseau. LightKrylov: Lightweight implementation of Krylov subspace techniques in modern Fortran. *Journal of Open Source Software*, 11(117):9623, 2026. doi: 10.21105/joss.09623.

[21] Mujeeb R. Malik. Numerical methods for hypersonic boundary layer stability. *Journal of Computational Physics*, 86(2):376–413, 1990.

[22] Bo Strand. Summation by parts for finite difference approximations for $d/dx$. *Journal of Computational Physics*, 110(1):47–67, 1994.

[23] Ken Mattsson and Jan Nordström. Summation by parts operators for finite difference approximations of second derivatives. *Journal of Computational Physics*, 199(2):503–540, 2004.

[24] Kamran Mohseni and Tim Colonius. Numerical treatment of polar coordinate singularities. *Journal of Computational Physics*, 157(2):787–795, 2000.

[25] Sanjiva K. Lele. Compact finite difference schemes with spectral-like resolution. *Journal of Computational Physics*, 103(1):16–42, 1992.

[26] Miguel R. Visbal and Datta V. Gaitonde. On the use of higher-order finite-difference schemes on curvilinear and deforming meshes. *Journal of Computational Physics*, 181(1): 155–185, 2002.

[27] Victorita Dolean, Pierre Jolivet, and Frédéric Nataf. *An Introduction to Domain Decomposition Methods: Algorithms, Theory, and Parallel Implementation*. SIAM, 2015.

[28] William Squire and George Trapp. Using complex variables to estimate derivatives of real functions. *SIAM Review*, 40(1):110–112, 1998.

[29] Joaquim R. R. A. Martins, Peter Sturdza, and Juan J. Alonso. The complex-step derivative approximation. *ACM Transactions on Mathematical Software*, 29(3):245–262, 2003.

[30] Alberto Padovan. resolvent4py example gallery: Cgl. `https://albertopadovan.github.io/resolvent4py/auto_examples/cgl/index.html`, 2026. Accessed 2026-03-01.